

Assignment - 3

Objective

Building a single CNN network with multi class classifier

heads for 3 different classification tasks on the same data

In this Assignment you are required to implement CNN in python to classify CIFAR 100 has 100 classes with 20 super classes

and each super class belong to the Group and there are 9 groups

So the model need to predict which class among 100 classes and which super class among 20 super class and which group among 9 groups

Solution

Here I'm doing comparison of 3 Architectures

I'm Applying same objective on 3 different models that is

- 1) CNN
- 2) Vision Transformer
- 3) DeiT [data efficient image transformer]

Preprocessing

• As per given in the question I written code such that from the dataset for each index I get the class, image and I created the dictionary where if I provide the class to the dictionary it gives the super class label. and I also created another dictionary where if I provide the super class label the dataset class gives the group label

• So finally for each index I get the image, image class, super class, Group Label

• image class there are 100 classes and 9 groups
super class there are 20 classes

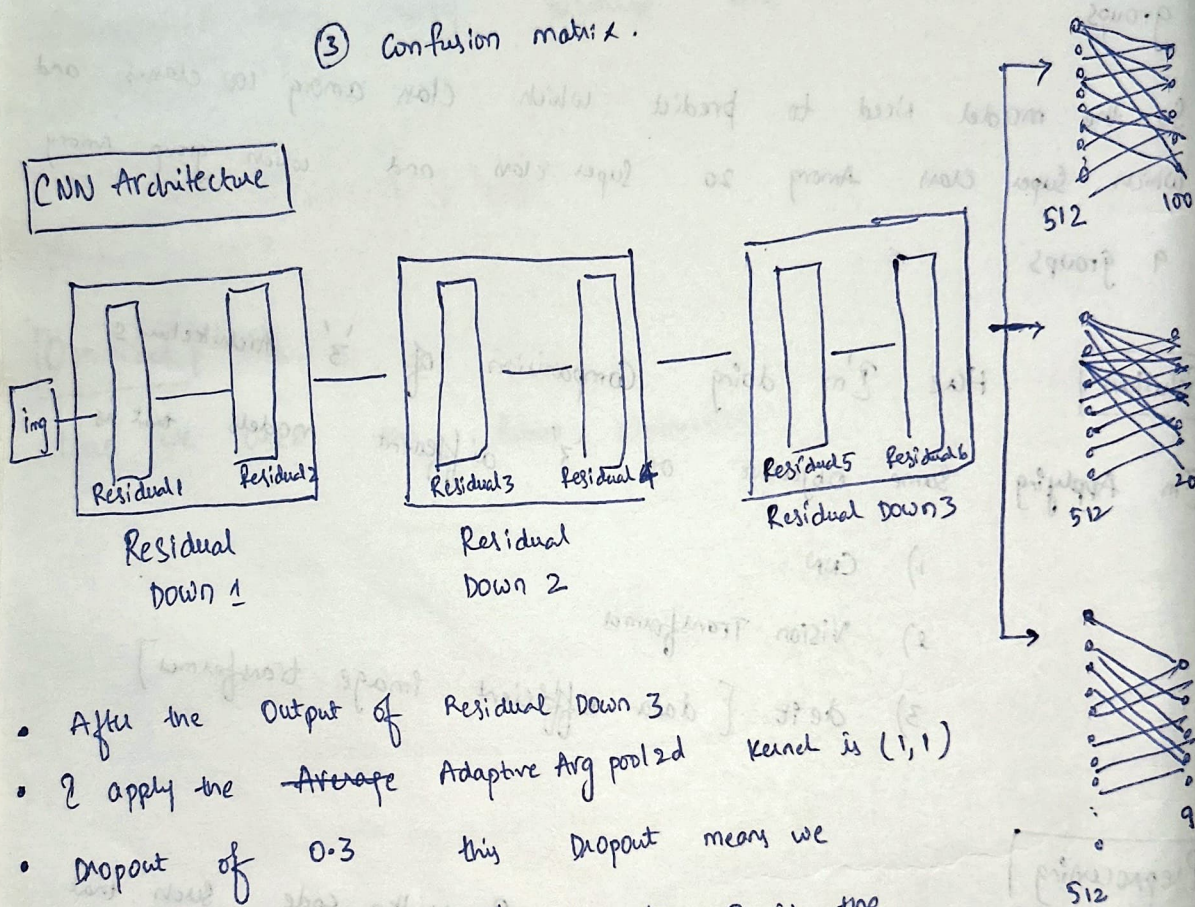
- I'm Training each model in 3 different splits

- 1) 70-30 Split
- 2) 80-20 Split
- 3) 90-10 Split

- Also plotting the

- ① Accuracy
- ② loss
- ③ Confusion matrix.

CNN Architecture



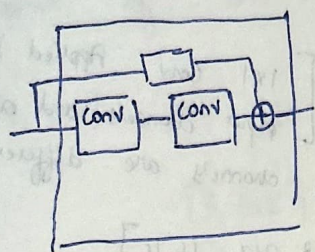
- After the Output of Residual Down 3
- I apply the ~~Average~~ Adaptive Arg pool 2d kernel is (1,1)
- Dropout of 0.3 this Dropout means we are going to drop the Neurons to Avoid the Overfitting
- And finally the Output of Dropout is Connected to 3 different fully Connected Network where each fully connected Network to predict

- ① Class label [So output layer contain 100 Neurons]
- ② Superclass label [So output layer contain 20 Neurons]
- ③ group label [So output layer contain 9 Neurons]

How the shape of the image flow in CNN let's see

Sol $[B, 3, 32, 32]$ is the tensor 2 get from the data loader

- Coming to Residual Down 1 It has 2 Residual Blocks
- Each Residual Block look like



Residual Block

Residual Connection is Applied. And added.

- So Residual 1

→ input_channel = 3 Given to the 1st Conv before the

- Residual down 1

→ $[B, 3, 32, 32] \rightarrow [B, 64, 32, 32]$

→ Now Residual Down 1 Starts from Here

Residual 1 { $[B, 64, 32, 32] \rightarrow [B, 64, 32, 32]$ [Conv 1 of Residual Block]
 $[B, 64, 32, 32] \rightarrow [B, 128, 32, 32]$ [Conv 2 of Residual Block]
 $[B, 64, 32, 32] \rightarrow [B, 128, 32, 32]$ [skip connection 1x1 conv]
 $[B, 128, 32, 32] + [B, 128, 32, 32] = [B, 128, 32, 32]$

Residual 2 { $[B, 128, 32, 32] \rightarrow [B, 128, 32, 32]$ [Conv 1 of Residual Block]
 $[B, 128, 32, 32] \rightarrow [B, 128, 32, 32]$ [Conv 2 of Residual Block]
 $[B, 128, 32, 32] + [B, 128, 32, 32]$ [Both Input and Output channels are of Same Shape So direct addition No Need of 1x1 Conv]

down Sample { $[B, 128, 32, 32] \rightarrow [B, 128, 16, 16]$

• Residual Down 2

Here also in Residual Down 2 contain 2 Residual Blocks
 & already shown how the Residual Block look like.

$$\begin{array}{l}
 \text{Residual 3} \left\{ \begin{array}{l}
 [B, 128, 16, 16] \rightarrow [B, 256, 16, 16] \quad [\text{Conv 1 of 3rd Residual}] \\
 [B, 256, 16, 16] \rightarrow [B, 256, 16, 16] \quad [\text{Conv 2 of 3rd Residual}] \\
 [B, 128, 16, 16] \xrightarrow[Conv]{1 \times 1} [B, 256, 16, 16] \quad [1 \times 1 \text{ Conv Applied because input channels and output channels are different}] \\
 [B, 256, 16, 16] + [B, 256, 16, 16] = [B, 256, 16, 16]
 \end{array} \right.
 \end{array}$$

$$\begin{array}{l}
 \text{Residual 4} \left\{ \begin{array}{l}
 [B, 256, 16, 16] \rightarrow [B, 256, 16, 16] \quad [\text{Conv 1 of 4th Residual}] \\
 [B, 256, 16, 16] \rightarrow [B, 256, 16, 16] \quad [\text{Conv 2 of 4th Residual}] \\
 [B, 256, 16, 16] + [B, 256, 16, 16] \quad [\text{No Need of } 1 \times 1 \text{ Conv because input \& output channels are same}] \\
 = [B, 256, 16, 16]
 \end{array} \right.
 \end{array}$$

$$\begin{array}{l}
 \text{down Sample} \left\{ \begin{array}{l}
 [B, 256, 16, 16] \rightarrow [B, 256, 8, 8]
 \end{array} \right.
 \end{array}$$

• Residual Down 3

In Residual Down 3 there are 2 Residual blocks they are
 Residual 5 Residual 6

$$\begin{array}{l}
 \text{Residual 5} \left\{ \begin{array}{l}
 [B, 256, 8, 8] \rightarrow [B, 512, 8, 8] \quad [\text{Conv 1 of 5th Residual}] \\
 [B, 512, 8, 8] \rightarrow [B, 512, 8, 8] \quad [\text{Conv 2 of 5th Residual}] \\
 [B, 256, 8, 8] \xrightarrow[Conv]{1 \times 1} [B, 512, 8, 8] \quad [\text{Both input \& output channels are different}] \\
 [B, 512, 8, 8] + [B, 512, 8, 8] = [B, 512, 8, 8]
 \end{array} \right.
 \end{array}$$

$$\begin{array}{l}
 \text{Residual 6} \left\{ \begin{array}{l}
 [B, 512, 8, 8] \rightarrow [B, 512, 8, 8] \quad [\text{Conv 1 of 6th Residual}] \\
 [B, 512, 8, 8] \rightarrow [B, 512, 8, 8] \quad [\text{Conv 2 of 6th Residual}] \\
 [B, 512, 8, 8] + [B, 512, 8, 8] = [B, 512, 8, 8]
 \end{array} \right.
 \end{array}$$

down
sample

$$[B, 512, 8, 1] \rightarrow [B, 512, 4, 4]$$

- Adaptive Average pool 2d $((1, 1)) \Rightarrow [B, 512, 4, 4] \rightarrow [B, 512, 1, 1]$
- Flatten $\Rightarrow [B, 512]$
- Dropout of 0.3 Applied
- $[B, 512]$ pass to fully connected of class label
 $[B, 512] \cdot [512, 100] = [B, 100]$
- $[B, 512]$ pass to fully connected of super class label
 $[B, 512] \cdot [512, 20] = [B, 20]$
- $[B, 512]$ pass to fully connected of group label
 $[B, 512] \cdot [512, 9] = [B, 9]$

	70-30 Split	80-20 Split	90-10 Split
Image label Accuracy	35%	40%	44%
Super label Accuracy	52%	57%	61%
Group label Accuracy	62%	68%	71%

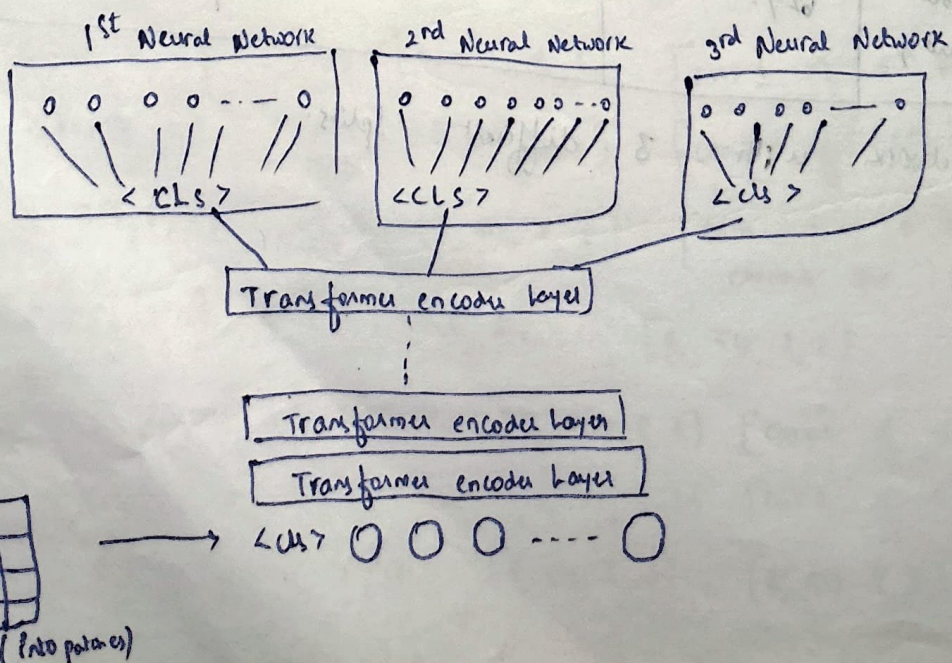
CNN Network with 3 different splits.

Vision Transformer

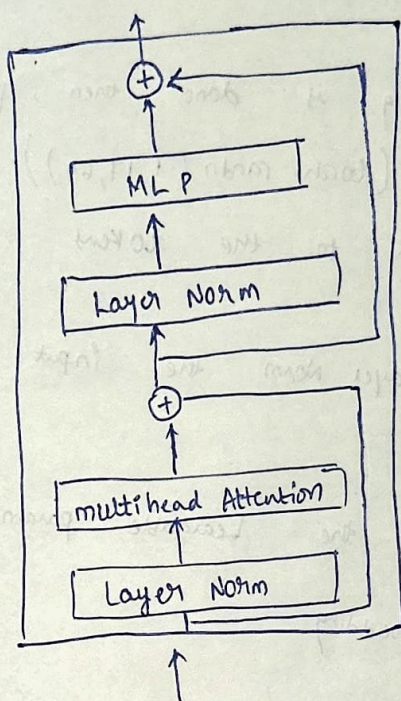
- Here I'm using Vision Transformer to classify the class label, Super class label, group label

Architecture of Vision Transformer

- In Vision Transformer image is divided into patches
- each patch converted to token and position information also added using the position embedding
- At the beginning $\langle \text{CLS} \rangle$ token is added such that this $\langle \text{CLS} \rangle$ token is used for the classification
- As the $\langle \text{CLS} \rangle$ token and the image patches pass through the Transformer Blocks finally $\langle \text{CLS} \rangle$ token taken and pass through '3' fully connected networks
- 1st fully connected network output layer contains 100 neurons where there are 100 class labels
- 2nd fully connected network output layer contains 20 neurons where there are 20 super class labels
- 3rd fully connected network output layer contains 9 neurons where there are 9 group labels



Transformer encoder Architecture



$\langle \text{cls} \rangle$ ○ ○ ○ ○ — ○

- Image is CIFAR-100 images so the size of the image is 32×32
- So from the Data loader the tensor I get is $[B, 3, 32, 32]$
- So this tensor is passed to the Patch embedding Block

$$\rightarrow [B, 3, 32, 32] \rightarrow \begin{array}{|l} \text{Conv 2D} \\ \text{in_channel} = 3 \\ \text{out_channel} = 64 \\ \text{kernel} = 8 \\ \text{Stride} = 8 \end{array} \rightarrow [B, 64, 4, 4]$$

$$\rightarrow [B, 64, 4, 4] \xrightarrow{\text{flatten}(2)} [B, 64, 16]$$

$$\rightarrow [B, 64, 16] \xrightarrow{\text{transpose}(1,2)} [B, 16, 64]$$

→ Each token is of size 64 dim

- Now $\langle \text{cls} \rangle$ token is Nothing But the nn.Parameter (torch.randn(1, 1, 64))

cls token is also of 64 dim

- $\langle U_i \rangle$ to Ken Concatenated with the Batch

$[B, 16, 64]$ and $[1, 1, 64]$ internally Broadcasting Applied

and $[B, 17, 64]$

- Now token embedding is done then position embedding which is nn.Parameter ($\text{torch.randn}(1, 17, 64)$) so position info is also Added to the tokens

Layer Norm TO the layer norm the input tensor is

$[B, 17, 64]$

So Assume γ, β are the Learnable parameters for one token embedding

$$x \in \mathbb{R}^D$$

$$\mu = \frac{1}{D} \sum_i x_i$$

$$\sigma^2 = \frac{1}{D} \sum_i (x_i - \mu)^2$$

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

ϵ very small value
denominator not to become 0

Then Learnable Scale & Shift.

$$y_i = \gamma \hat{x}_i + \beta$$

If model want to move to some other distribution rather than the standard Normal distribution we can use those trainable parameters.

- The Output of layer Norm Shape $[B, 17, 64]$

Multihead Attention

- This is not the Causal Attention Applied this means the current token has the Access of the future tokens

- Assume the input to the Multihead Attention is $[B, T, D]$

B = Batch Size

T = Num of Tokens

D = Token Dimension

- W_Q, W_K, W_V are the Learnable matrices

$$Q = X \cdot W_Q$$

$$K = X \cdot W_K$$

$$V = X \cdot W_V$$

$$Q, K, V \in \mathbb{R}^{B \times T \times D}$$

- Split into multiple heads

Suppose embedding-dim (D) = 512

num of heads (h) = 8

$$d_k = 512 / 8 = 64 \quad \text{i.e. } D/h$$

$$\text{Reshape } [B, T, D] \Rightarrow [B, h, T, d_k]$$

- Compute Attention scores of each head

$$Q \cdot K^T$$

$$[B, h, T, d_k] \cdot [B, h, d_k, T]$$

$$= [B, h, T, T]$$

- Scale the Scores to Stabilize gradients

$$\frac{Q \cdot K^T}{\sqrt{d_k}}$$

why we are doing $\sqrt{d_k}$
becoz Variance. Need to stay close to 1 otherwise Softmax will assign High probability to Higher value

• Apply Softmax

$$A = \text{Softmax} \left(\frac{Q \cdot K^T}{\sqrt{d_k}} \right)$$

The shape of $A = [B, h, T, T]$

$$Z = A \cdot V$$

$$= [B, h, T, T] \cdot [B, h, T, d_k]$$

$$= [B, h, T, d_k]$$

• Concat heads

$$[B, h, T, d_k] \Rightarrow [B, T, D]$$

• final Linear projection

$$\text{Output} = Z \cdot W_o$$

$$= [B, T, D]$$

• Now the Residual Connection

$$[B, T, D] + [B, T, D] = [B, T, D]$$

$$(\text{i.e.}) [B, 16, 64]$$

• Again Layer norm shape Remains Same

$$[B, 16, 64]$$

• MLP

$$[B, 16, 64] \cdot [64, 256] = [B, 16, 256]$$

$$[B, 16, 256] \cdot [256, 64] = [B, 16, 64]$$

Projected to 4 times 64 dim = 256 dim and

GELU()

GELU()

$x \cdot \phi(x)$

of standard normal

where $\phi(x)$ is the Cumulative Distribution function

• Output of MLP is $[8, 16, 64]$

• Residual Connection is Added $[8, 16, 64] + [8, 16, 64]$

• Finally $\langle \text{cls} \rangle$ token is fetched in the Batch By

$[:, 0, :]$

• And pass through 3 Fully Connected Network

1st $\rightarrow$ class label

2nd $\rightarrow$ Super class label

3rd $\rightarrow$ group label.

Loss function

This is multi class classification

$$L = - \sum y_i \log \hat{y}_i$$

Parameters

1) batch_size = 64

2) epochs = 100

3) $\eta = 1e-3$

4) weight decay = $1e-4$

5) patch_size = 8

6) attention_heads = 4

7) transformer_blocks = 4

8) MLP_hidden_nodes = 256

9) img_size = 32

10) embedding_dim = 64

11) num_patches =
 $(\text{img_size} // \text{patch_size})^2 + 2$

16 patches

12) Adam Optimizer

$\beta_1 = 0.9, \beta_2 = 0.999$

• We Ran Vit for 70-30, 80-20, 90-10 Splits

Accuracy vs Splits	70-30	80-20	90-10
Image label Accuracy	0.24 = 24%	25 %	24 %
Super class label	39%	40%	38 %
group label	50%	51 %	50%

DEIT

[Data efficient Transformer]

• Here Knowledge Distillation is performed

→ Teacher is Nothing But the Resnet50

→ Student is the Transformer like the vision transformer

But extra token is appended at the end

→ $\langle \text{cls} \rangle$ token at the beginning and the
 $\langle \text{distill} \rangle$ token at the end and in between
 token embedding of the patches

• First I will fine tune the Teacher Resnet 50 on my
 CIFAR-100 Dataset

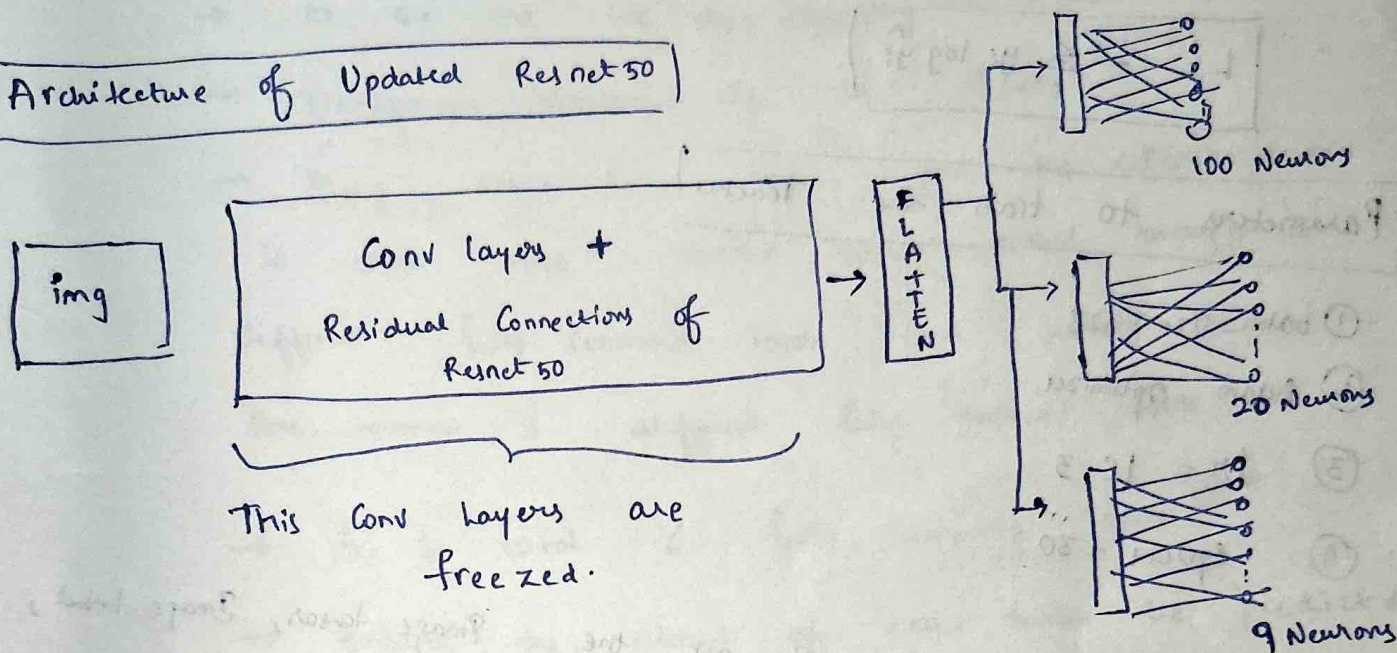
• All the Convolution layers are frozen But I will Remove
 the Fully Connected layer of Resnet50 And the Result of
 last Convolution layer and we Flatten it and we pass that
 flattened tensor to 3 fully Connected layers

• 1st fully Connected layer take the flattened tensor
 and O/p layer has 100 Neurons

- 2nd fully connected layer take the flattened tensor and o/p layer contain 20 Neurons

- 3rd fully Connected layer take the flattened tensor and o/p layer contain 9 Neurons.

Architecture of Updated Resnet 50



- So img pass through the Resnet 50

$$[B, 3, 32, 32] \xrightarrow[\text{flatten}]{\text{After}} [B, 2048]$$

- 1st fully Connected has the weight matrix of $[2048, 100]$

$$[B, 2048] \cdot [2048, 100] = [B, 100]$$

- 2nd fully Connected has the weight matrix of $[2048, 20]$

$$[B, 2048] \cdot [2048, 20] = [B, 20]$$

- 3rd fully Connected has the weight matrix of $[2048, 9]$

$$[B, 2048] \cdot [2048, 9] = [B, 9]$$

Loss ÷

It is the multi class classification so Categorical cross entropy
on '3' of them is applied and loss is Added and
Backprop the Loss

$$L = - \sum_i y_i \log \hat{y}_i$$

Parameters to train the Teacher

① batch-size = 128

② Adam optimizer

③ $lr = 1e-3$

④ epochs = 30

- from the DataLoader I get the Image tensor, Image label, Super class label, Group label.
- I pass image tensor to PerNet 50 and pass flattened tensor to '3' fully connected layers
- I have the True label & predicted label and Apply the Categorical Cross entropy
- then Backprop till the fully connected layers Because I freeze the Conv layers of PerNet 50

Student Transformer model

The model is same Architecture as the Vision Transformer But the changes done in the Architecture

→ At the end we are appending the $\langle \text{distill} \rangle$ token

→ Transformer Architecture is the same

→ finally $\langle \text{cls} \rangle$ token taken and the $\langle \text{distill} \rangle$ token is taken then $\langle \text{cls} \rangle$ token passed through 3

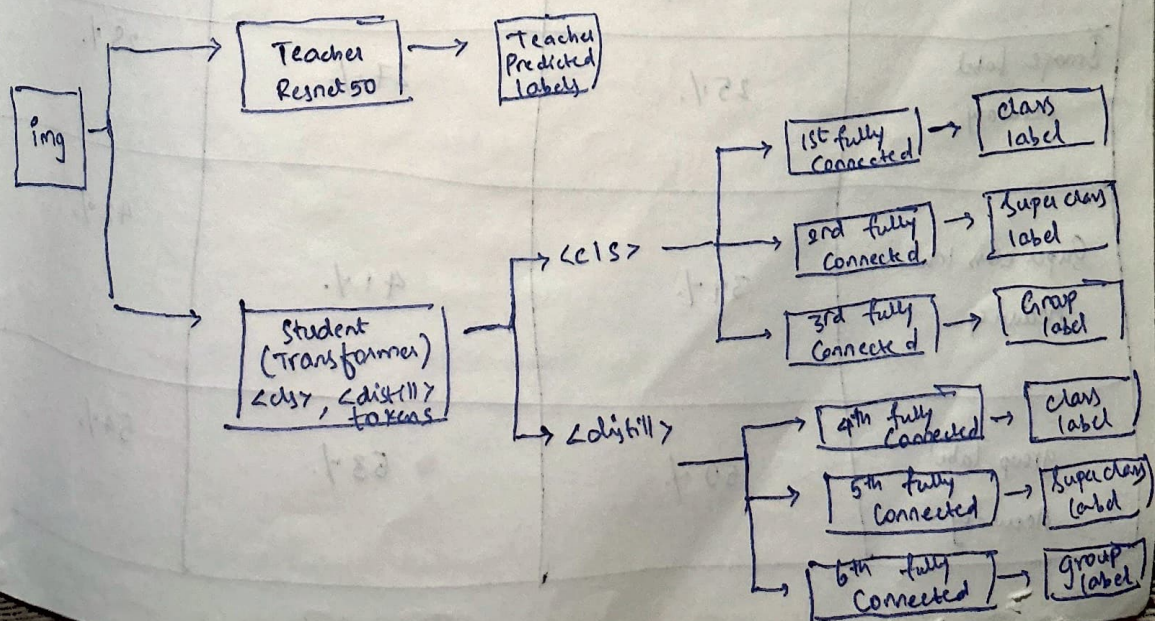
different fully connected layers and $\langle \text{distill} \rangle$ token

Pass through 3 different fully connected layers

→ So in total 6 fully connected layers

→ Between the output of $\langle \text{cls} \rangle$ token i.e. predicted labels and actual label categorical cross entropy used

→ Between the output of $\langle \text{distill} \rangle$ token i.e. predicted labels by the distill token and one thing while training the student we freeze the ~~transformer~~ Teacher and output of Teacher labels and output labels of distill token KL divergence is Applied.



KL Divergence

How close the predicted Probability distribution to the Original probability distribution

- $KL(P \parallel Q) = \sum P(x) \log \frac{P(x)}{Q(x)}$

- KL is Always ≥ 0

- When 2 probability distributions are close then $KL = 0$ else $KL > 0$

- Transformer Architecture is exactly the same

But extra $\langle \text{distill} \rangle$ token is Added Here.

- Convert image into patches and patches converted to tokens

- The input to transformer is $[B, 18, 64]$

Actually 16 patches But 2 tokens $\langle \text{cls} \rangle, \langle \text{distill} \rangle$ so total 18

- Also position info is also Added Here.

	70-30 Split	80-20 Split	90-10 Split
Image label Accuracy	25%.	27%.	28%
Super class label Accuracy	39%.	41%.	42%
group label Accuracy	50%.	53%.	54%

Parameters used for DEET

batch size = 128

epochs = 100

lr = $3e-4$

weight-decay = $1e-4$